

ESP-IDF GPIO, ADC & PWM Fundamentals

*Potentiometer-Driven PWM Brightness and IR-Sensor Digital Switching on a
Bare ESP-IDF ESP32 Application*



ESPRESSIF

*Dana Natsheh
Cyber Robot
IoT*

1. Executive Summary

This report documents Task 3.4: implementing the three foundational ESP32 peripheral APIs – digital GPIO, ADC, and LEDC PWM – using pure ESP-IDF, with no Arduino compatibility layer and no RTOS tasks beyond the single implicit task ESP-IDF itself requires for `app_main`. Two independent input-to-output chains demonstrate the three APIs together: a potentiometer, read through the `adc_oneshot` driver, sets the PWM duty cycle (via `ledc`) of one LED; an IR obstacle sensor, polled through plain `gpio_get_level`, switches a second LED fully on or off.

The bulk of this task's effort, and its most instructive content, was not the peripheral code itself – each driver call is a handful of lines – but two separate debugging journeys: getting a working ESP-IDF build environment on Windows at all, and diagnosing a PWM brightness fault that looked like ADC noise but was actually a marginal physical connection. Both are documented in full in Section , since both produced misleading initial symptoms that pointed toward the wrong fix before the real cause was found.

2. System Overview & Goal

```
(Potentiometer wiper) --adc_oneshot_read--> [ESP32 ADC1] --scale 12-bit to 13-bit--> [LEDC PWM] --> LED1
                                     brightness
(IR sensor OUT) --gpio_get_level, polled--> [ESP32 GPIO input] --if/else--> [gpio_set_level] --> LED2
                                     on/off
```

- Both chains run inside a single `while(1)` polling loop in `app_main`, read once per iteration, and are separated purely by which peripheral driver each block calls – there is no task, queue, or interrupt separating them.
- The potentiometer chain is continuous: any wiper position maps to a proportional LED brightness.
- The IR sensor chain is binary: the sensor's digital output is read directly and mirrored onto the second LED with no intermediate state.
- A fixed 50 ms `vTaskDelay` paces the loop, giving both chains an effective update rate of roughly 20 Hz.

3. Hardware Architecture

Pin	Peripheral	Role
GPIO2	LEDC PWM channel 0, timer 0	Brightness LED, driven by the potentiometer reading
GPIO4	Plain digital output	Second LED, mirrors the IR sensor's state
GPIO5	Plain digital input	IR obstacle sensor output, polled every loop iteration
GPIO34	ADC1 channel 6	Potentiometer wiper; input-only pin, no internal pull resistor available

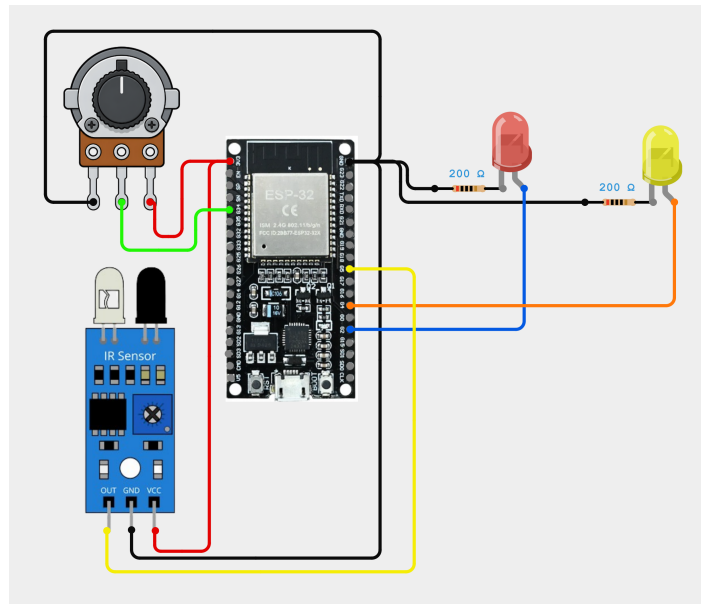


Figure 1: wiring schematic

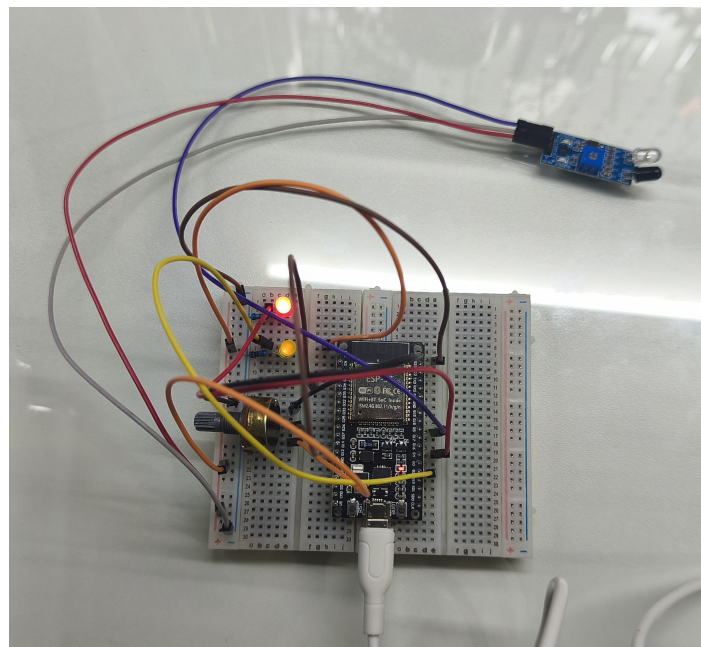


Figure 2: Assembled breadboard circuit.

Power wiring is identical across all four peripherals: every module's VCC/+ leg returns to the ESP32's 3V3 rail and every GND/− leg returns to GND. The PWM and digital LEDs each sit in series with a 220–330 Ω current-limiting resistor. GPIO34 was deliberately chosen for the potentiometer specifically because it is a genuine ADC-only pin on the classic ESP32 – a constraint that later became directly relevant to the debugging journal in Section .

4. Development Environment & Toolchain Setup

- Target ESP-IDF version: v5.0 or later, required by the `adc_oneshot` driver used in this task (the older `adc1_get_raw`-style API is not used here).
- Initial setup used the official Espressif IDF extension for VS Code (ESP-IDF: New Project wizard), targeting `esp32`.
- After repeated build friction on this machine (documented in Section), the project was migrated to PlatformIO (`platform = espressif32`, `framework = espidf`, `board = esp32dev`), which uses a `src/main.c` + `platformio.ini` layout instead of ESP-IDF's native `main/` + root `CMakeLists.txt` layout. Application source code required no changes to move between the two.

```
1 [env:esp32dev]
2 platform = espressif32
3 board = esp32dev
4 framework = espidf
5 monitor_speed = 115200
```

Listing 1: platformio.ini

5. Firmware Implementation

GPIO setup

```
1 static void gpio_setup(void)
2 {
3     gpio_set_direction(IR_LED_PIN, GPIO_MODE_OUTPUT);
4     gpio_set_level(IR_LED_PIN, 0);
5
6     gpio_set_direction(IR_SENSOR_PIN, GPIO_MODE_INPUT);
7 }
```

Listing 2: Digital output LED and digital input IR sensor

ADC oneshot setup

```

1 static adc_oneshot_unit_handle_t adc_setup(void)
2 {
3     adc_oneshot_unit_handle_t adc1_handle;
4
5     adc_oneshot_unit_init_cfg_t init_config = {
6         .unit_id = POT_ADC_UNIT,
7     };
8     ESP_ERROR_CHECK(adc_oneshot_new_unit(&init_config, &adc1_handle));
9
10    adc_oneshot_chan_cfg_t chan_config = {
11        .bitwidth = POT_ADC_BITWIDTH,
12        .atten     = POT_ADC_ATTEN,
13    };
14    ESP_ERROR_CHECK(adc_oneshot_config_channel(adc1_handle, POT_ADC_CHANNEL, &
15        chan_config));
16
17    return adc1_handle;
18 }

```

Listing 3: Potentiometer ADC channel configuration

ADC_ATTEN_DB_12 extends the ADC's usable input range to approximately the full 0–3.3V rail; without it, the ADC's native ~1.1 V ceiling would clip the top portion of the potentiometer's travel.

LEDC PWM setup

```

1 static void pwm_setup(void)
2 {
3     ledc_timer_config_t timer_conf = {
4         .speed_mode     = LEDC_MODE,
5         .timer_num      = LEDC_TIMER,
6         .duty_resolution = LEDC_DUTY_RES,
7         .freq_hz        = LEDC_FREQUENCY,
8         .clk_cfg        = LEDC_AUTO_CLK,
9     };
10    ESP_ERROR_CHECK(ledc_timer_config(&timer_conf));
11
12    ledc_channel_config_t channel_conf = {
13        .gpio_num    = PWM_LED_PIN,
14        .speed_mode  = LEDC_MODE,
15        .channel     = LEDC_CHANNEL,
16        .timer_sel   = LEDC_TIMER,
17        .duty        = 0,
18        .hpoint      = 0,
19    };
20    ESP_ERROR_CHECK(ledc_channel_config(&channel_conf));
21 }

```

Listing 4: PWM timer and channel configuration

Main polling loop

```

1 void app_main(void)
2 {
3     gpio_setup();
4     adc_oneshot_unit_handle_t adc1_handle = adc_setup();

```

```

5   pwm_setup();
6
7   while (1) {
8       int samples[POT_SAMPLES];
9       for (int i = 0; i < POT_SAMPLES; i++) {
10          ESP_ERROR_CHECK(adc_oneshot_read(adc1_handle, POT_ADC_CHANNEL, &samples[i]
11          ));
12      }
13      for (int i = 1; i < POT_SAMPLES; i++) {
14          int key = samples[i];
15          int j = i - 1;
16          while (j >= 0 && samples[j] > key) {
17              samples[j + 1] = samples[j];
18              j--;
19          }
20          samples[j + 1] = key;
21      }
22      int raw = samples[POT_SAMPLES / 2];
23
24      uint32_t duty = (raw * 8191) / 4095;
25      ledc_set_duty(LEDc_MODE, LEDc_CHANNEL, duty);
26      ledc_update_duty(LEDc_MODE, LEDc_CHANNEL);
27
28      int ir_state = gpio_get_level(IR_SENSOR_PIN);
29      bool object_detected = (ir_state == IR_ACTIVE_LEVEL);
30      gpio_set_level(IR_LED_PIN, object_detected ? 1 : 0);
31
32      vTaskDelay(pdMS_TO_TICKS(50));
33  }

```

Listing 5: app_main – median-filtered ADC read, PWM update, IR-driven digital output

6. Polling Loop Behavior

No FreeRTOS tasks, queues, or semaphores are created by this application; `app_main` itself runs as the single task ESP-IDF's startup code always creates, and `vTaskDelay` is used purely as a sleep primitive to pace the loop, not as a synchronization mechanism between concurrent work. Both the ADC read block and the IR sensor check execute sequentially within the same iteration, so the IR sensor's worst-case response latency is bounded by the loop period (here, roughly 50ms plus the time to take 16 ADC samples) rather than by any scheduler decision. This is adequate for two peripherals updated at human-interaction speed, but does not scale cleanly to additional concurrent peripherals without either shortening the ADC sampling block or introducing genuine task-based concurrency in a later task.

7. ADC-to-PWM Scaling

The ADC oneshot driver returns a 12-bit raw value (0–4095) at the configured bit width; the LEDC timer in this task is configured for 13-bit duty resolution (0–8191). The two ranges are linked with a single linear scale:

$$\text{duty} = \left\lfloor \frac{\text{raw} \times 8191}{4095} \right\rfloor$$

Matching the ADC bit width and LEDC duty resolution (e.g. both at 12-bit) would remove the need for this line entirely; the mismatch was kept deliberately in this implementation to demonstrate the scaling step explicitly.

8. Debugging Journal

8.1 Toolchain and Build Failures

Symptom 1: nested project fails to configure

The first build attempt, using the VS Code ESP-IDF extension’s generated project, failed during CMake configuration with `project command is not scriptable`, originating from `components/esp-idf/CMakeLists.txt` inside the project folder itself.

Root cause. ESP-IDF’s build system treats every folder under `components/` as a component to compile alongside `main/`. A full copy of ESP-IDF’s own top-level `CMakeLists.txt` – which itself calls `project()` – had ended up nested inside `components/esp-idf/`, so CMake attempted to process a second, illegal `project()` call partway through configuring the first.

Fix. The stray `components/esp-idf/` folder was deleted (ESP-IDF is installed once, separately, and located via `IDF_PATH` – it does not belong inside an individual project’s `components/` directory), followed by a full clean of the stale `build/` directory.

Symptom 2: PlatformIO venv creation fails with WinError 32

After migrating to PlatformIO to sidestep further friction with the native ESP-IDF CMake project layout, the first `espidf` framework build failed with:

```
1 Error: [WinError 32] The process cannot access the file because it is
2 being used by another process:
3 'C:\Users\User\.platformio\penv\espidf-6.0.1\pyvenv.cfg'
```

Listing 6: WinError 32 during Python virtual environment creation

Root cause. PlatformIO’s first-time `espidf` framework setup creates a dedicated Python virtual environment. On Windows, real-time antivirus scanning (or a leftover process from a previously interrupted setup attempt) held a lock on a file inside that environment while it was being written, causing venv creation to abort partway through.

Fix. All VS Code windows/terminals and any orphaned `python.exe/pio.exe` processes were closed, the partially-created `.espidf-6.0.1` venv folder was deleted, and `C:\Users\User\.platformio` was added as a Windows Defender exclusion to prevent the same class of failure recurring on future package installs.

Symptom 3: CMake requirements parser fails on IDF's own code

The next build attempt progressed further, then failed with:

```
1 CMake Error: ERROR: Cannot parse CMake requirements line:
2   idf_build_get_property(__cm_sdkconfig SDKCONFIG)
```

Listing 7: CMake requirements parsing failure

Root cause. ESP-IDF's build system uses a lightweight, line-based text parser (not full CMake evaluation) to extract each component's `REQUIRES/PRIV_REQUIRES` declarations early in configuration. This parser choked on a line from ESP-IDF's own internal build machinery, consistent with a partially or corruptly downloaded `framework-espidf` package – plausible given that the prior venv-lock failure had interrupted the same overall install sequence.

Fix. The `build/` directory was deleted, and the entire `framework-espidf` package directory was removed from PlatformIO's package cache to force a clean re-download on the next build, which resolved the parsing failure.

Symptom 4: misleading nested-function compile errors

With the toolchain finally stable, the first source-code build failed with:

```
1 src/main.c:37:34: error: invalid storage class for function 'adc_setup'
2 src/main.c: In function 'gpio_setup':
3 ...
4 src/main.c:107:1: error: expected declaration or statement at end of input
```

Listing 8: GCC output – symptom, not root cause

Root cause. `gpio_setup()` was missing its closing brace, so the compiler treated every subsequent function definition (`adc_setup`, `pwm_setup`) as nested inside it – which C does not permit, hence “invalid storage class.” A second brace was also missing at the very end of the file, where only one closing brace was present to close both the `while(1)` loop and `app_main` itself. The reported error locations (inside `gpio_setup`, and at the file's last line) were artifacts of where the compiler's brace-counting finally ran out, not the actual location of the missing braces.

Fix. A closing `}` was inserted immediately after `gpio_setup`'s body, and a second closing `}` was added at the end of the file to separately close the `while` loop and `app_main`.

8.2 The Blinking Potentiometer

Symptom

With the firmware flashed and running, turning the potentiometer to its maximum position caused the PWM LED to visibly flicker rather than hold a steady maximum brightness.

Hypothesis 1: ADC noise near the supply rail (partially addressed, not the root cause)

The ESP32's ADC is known to be noisier near the top of its input range, particularly under `ADC_ATTEN_DB_12`. A 16-sample oversampled mean was added to the potentiometer read, smoothing the kind of small jitter this hypothesis predicts.

Result. Flicker persisted. Serial log inspection showed the actual fault pattern was not small jitter around a stable value – readings were clustered tightly (e.g. 2203, 2205, 2203, 2206) during normal operation, then, intermittently, alternated cleanly between the absolute extremes on consecutive loop iterations:

```
1 Pot raw=4095 duty=8191 | IR=1 -> LED=OFF
2 Pot raw=0      duty=0   | IR=1 -> LED=OFF
3 Pot raw=4095 duty=8191 | IR=1 -> LED=OFF
4 Pot raw=0      duty=0   | IR=1 -> LED=OFF
```

Listing 9: Serial log excerpt – clean 0/4095 alternation, not noise

This pattern – clean alternation between minimum and maximum, rather than jitter around the true wiper position – is the signature of a floating or intermittently disconnected input, not ADC noise.

Hypothesis 2 (considered and rejected): add a pull-up resistor

A pull-up on the ADC pin was considered as a fix for a floating input.

Rejected for two reasons. First, GPIO34 is one of the ESP32's input-only pins (GPIO34–39); Espressif's silicon omits the internal pull-up/pull-down circuitry on this pin range entirely, so `gpio_set_pull_mode` on GPIO34 returns `ESP_ERR_NOT_SUPPORTED` rather than doing anything. Second, even where a pull resistor is physically possible, it is the wrong tool for a continuously variable analog signal: a pull-up biases the pin toward a fixed rail, which fights against a potentiometer's legitimate mid-range voltages, and would only mask an intermittent open connection by making it default to a steady 4095 rather than surfacing the fault.

Hypothesis 3 (the real root cause): marginal physical connection

The clean 0/4095 alternation, combined with the fact that the fault was most noticeable near the mechanical end of the potentiometer's travel (where wire stress from turning the knob is highest), pointed to a loose or marginal connection at the wiper wire – either at the breadboard contact or the jumper wire itself – rather than a code or silicon issue.

Fix

Two complementary changes were made:

1. **Physical.** A wiggle-test procedure (observing raw ADC values on the serial monitor while gently disturbing each connection point in turn) was used to isolate the specific bad contact, followed by reseating the wiper wire at both ends and, where needed, relocating to an unused breadboard row.
2. **Software.** The oversampled *mean* was replaced with a *median* filter over the same 16 samples. A mean is pulled toward any single outlier in the sample set; a median ignores an outlier entirely as long as it does not make up more than half the samples, which directly targets this fault's signature of occasional extreme spikes among otherwise-good readings.

```
1 int samples[POT_SAMPLES];
2 for (int i = 0; i < POT_SAMPLES; i++) {
3     ESP_ERROR_CHECK(adc_oneshot_read(adc1_handle, POT_ADC_CHANNEL, &samples[i]));
4 }
5 /* insertion sort, then take the middle element */
6 for (int i = 1; i < POT_SAMPLES; i++) {
7     int key = samples[i];
8     int j = i - 1;
9     while (j >= 0 && samples[j] > key) {
10         samples[j + 1] = samples[j];
11         j--;
12     }
13     samples[j + 1] = key;
14 }
15 int raw = samples[POT_SAMPLES / 2];
```

Listing 10: Median filter – robust to occasional 0/4095 spikes, unlike a mean

Confirmation

Takeaway: the initial fix (oversampled averaging) was a reasonable first guess that treated the symptom as noise, and it was only ruled out by actually reading the raw serial log rather than just observing that the LED still flickered. A mean and a median can look interchangeable for well-behaved noisy data, but they respond completely differently to occasional extreme outliers – which is exactly the kind of distinction that matters when a fault is intermittent and physical rather than continuous and electrical. Verification of full hardware stability (i.e. confirming the loose connection itself is permanently resolved, not just statistically masked) was still in progress at the time of writing.

9. Results & Verification

Potentiometer sweep tracks PWM duty linearly

With the median filter applied, sweeping the potentiometer from minimum to maximum produced a smooth, monotonic duty-cycle trace in the serial log, free of the single-sample spikes seen prior to filtering.

```

I (10662) TASK3_4: Pot raw=1855 duty=3710 | IR=0 -> LED=ON
I (10712) TASK3_4: Pot raw=1921 duty=3842 | IR=0 -> LED=ON
I (10762) TASK3_4: Pot raw=1932 duty=3864 | IR=0 -> LED=ON
I (10812) TASK3_4: Pot raw=1914 duty=3828 | IR=0 -> LED=ON
I (10862) TASK3_4: Pot raw=1856 duty=3712 | IR=0 -> LED=ON
I (10912) TASK3_4: Pot raw=1768 duty=3536 | IR=0 -> LED=ON
I (10962) TASK3_4: Pot raw=1664 duty=3328 | IR=0 -> LED=ON
I (11012) TASK3_4: Pot raw=1578 duty=3156 | IR=1 -> LED=OFF
I (11062) TASK3_4: Pot raw=1452 duty=2904 | IR=1 -> LED=OFF
I (11112) TASK3_4: Pot raw=1335 duty=2670 | IR=1 -> LED=OFF
I (11162) TASK3_4: Pot raw=1217 duty=2434 | IR=1 -> LED=OFF
I (11212) TASK3_4: Pot raw=1107 duty=2214 | IR=1 -> LED=OFF

```

Figure 3: Serial monitor output during a full potentiometer sweep: raw ADC value, scaled PWM duty, IR sensor state, and resulting digital LED state, logged once per loop iteration.

IR sensor reliably switches the second LED

Presenting an object to the IR sensor was confirmed to flip `ir_state` and, in the same loop iteration, switch the digital LED, independent of whatever the potentiometer was doing at the same time – confirming the two peripheral chains do not interfere with each other despite sharing a single polling loop.

10. Design Notes – Efficient Choices

1. **One sample buffer serves both smoothing and outlier rejection.** Rather than layering a separate noise filter on top of a separate spike detector, a single 16-sample median filter does both jobs at once, since a median is simultaneously a smoothing operator and a robust-statistics outlier rejector.
2. **ADC-only pin chosen deliberately for the potentiometer.** Using GPIO34 rather than a general-purpose GPIO for the analog input keeps the pin unambiguously dedicated to ADC1, at the direct cost – surfaced by the debugging journal – of having no internal pull resistor available as a software safety net.
3. **Named peripheral constants over bare pin numbers.** Using `GPIO_NUM_4` rather than the literal 4 costs nothing at runtime (C enums compile to plain integers) but gains compile-time validation against the target chip's actual pin range and self-documents intent at each call site.
4. **Explicit linear scaling between mismatched bit widths.** Rather than matching the ADC and LEDC resolutions to avoid scaling entirely, the 12-bit-to-13-bit conversion is kept as a single explicit line, making the relationship between raw ADC counts and PWM duty visible rather than implicit.

5. **Pure polling kept honestly simple.** With only two peripherals and no timing-critical response requirement, a flat `while(1)` loop with one `vTaskDelay` avoids introducing RTOS primitives before the task actually calls for them.

11. Known Limitations & Considerations

- **Polling latency is bounded by loop period, not peripheral response time.** The IR sensor's effective response time is the loop's ~ 50 ms period plus 16 back-to-back ADC reads, not the sensor's own (much faster) electrical response – acceptable here, but a ceiling that a purely polled design will keep hitting as more peripherals are added.
- **Median filter cost scales with sample count.** The insertion sort used to find the median is $O(n^2)$, negligible at $n = 16$ but not a filter that should be scaled up to a much larger sample count without switching to a faster selection algorithm.
- **Physical fix not independently confirmed stable long-term.** The median filter statistically tolerates the intermittent fault; it does not verify the underlying connection has been permanently repaired.

12. Bill of Materials

Item	Qty	Notes
ESP32 DevKit development board	1	Actual onboard flash detected as 2 MB during build
LED	2	One PWM-driven (GPIO2), one digital on/off (GPIO4)
Current-limiting resistor	2	220–330 Ω , one per LED
IR obstacle sensor module	1	3-pin digital output (VCC, GND, OUT)
Potentiometer	1	Wiper on GPIO34 (ADC1 channel 6)
Breadboard	1	
Jumper wires	Assorted	Including the specific wiper-to-GPIO34 wire implicated in Section
USB cable	1	Flashing and serial monitor
PC running VS Code + ESP-IDF extension and/or PlatformIO Core	1	ESP-IDF v5.x+ toolchain

13. Future Improvements

- **Interrupt-driven IR sensing** once RTOS tasks are introduced in a later task, replacing fixed-period polling with an ISR-plus-queue design that reacts immediately rather than on the next loop tick.
- **Replace the flat median filter with an exponential moving average or lightweight Kalman filter**, once the underlying physical connection fault is fully confirmed resolved, trading some outlier robustness back for lower per-loop computation and smoother transient response.
- **Perceptual (gamma-corrected) brightness mapping** instead of linear ADC-to-duty scaling, since human brightness perception is non-linear and a linear duty sweep feels front-loaded at low pot positions.
- **Resolve the 2 MB vs. 4 MB flash size mismatch** explicitly in `platformio.ini`, rather than leaving it as a passive build warning.
- **Add a small hardware decoupling capacitor** between the potentiometer wiper and GND as a permanent, hardware-level complement to the software median filter, independent of whether every marginal connection has been physically found.